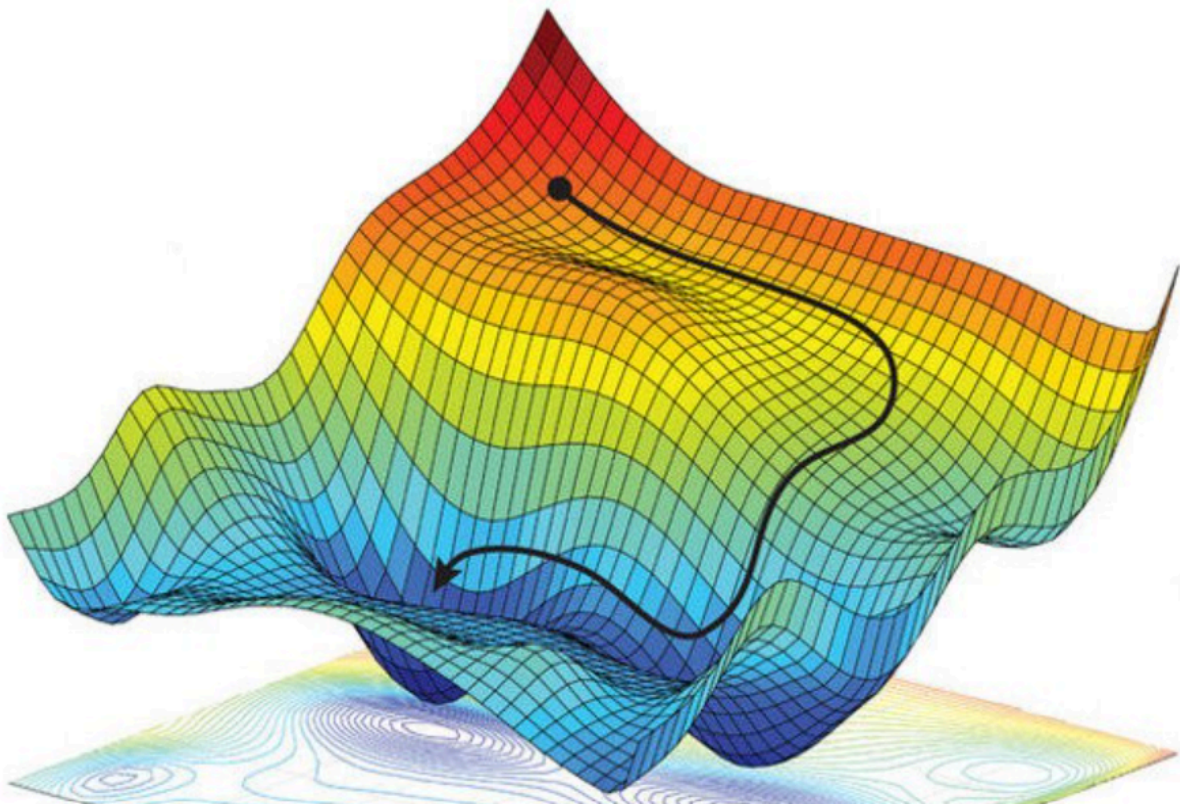


Task 4 Group Report: Deterministic vs Stochastic Optimization

Eneko ORHATEGARAY, Paul BROCVIELLE, Aiert CECCON, Achille LARREGLE

Date: 2026-01-19



1. Introduction.....	3
2. Methodology: BFGS & Penalty Functions.....	4
2.1 The BFGS Update Formula.....	4
2.2 Python Implementation.....	4
3. Experimental Setup.....	5
3.1 Test Functions.....	5
3.2 Comparison Protocol.....	8
4. Results and Analysis.....	9
4.1 Unimodal Efficiency (Rosenbrock).....	9
4.3 Constrained Optimization (Rosenbrock).....	11
4.2 Multimodal Trap (Rastrigin).....	13
4.3 Constrained Optimization (Penalty Analysis).....	15
5. Discussion.....	16
5.1 Efficiency vs Robustness.....	16
5.2 Hybrid Strategy Proposal.....	16
5.3 Limitations.....	16
6. Conclusion.....	17
7. Individual Contribution & Indicative Marks.....	18
Summary of Workload Distribution.....	19

1. Introduction

The primary objective of this study is to investigate the performance of a deterministic gradient-based optimization method and to compare it against the stochastic metaheuristic algorithms previously evaluated in Task 3. In this context, this report examines the Broyden-Fletcher-Goldfarb-Shanno (BFGS) algorithm, a Quasi-Newton method that represents a distinct approach to mathematical optimization.

Building upon the findings of Task 3, it was observed that Particle Swarm Optimization (PSO) and Simulated Annealing (SA) were effective on multimodal landscapes characterized by multiple local optima. However, these stochastic methods necessitate a substantial number of function evaluations to converge, even on simpler unimodal problems. In contrast, gradient-based methods offer an alternative strategy by explicitly following the direction of steepest descent, utilizing local curvature approximations to accelerate convergence.

As a Quasi-Newton method, BFGS approximates the inverse Hessian matrix at each iteration. This allows it to achieve super-linear convergence rates on smooth, twice-differentiable functions, thereby avoiding the high computational cost associated with calculating exact second derivatives. Consequently, BFGS is theoretically more efficient when the objective function is smooth and the landscape is unimodal.

The central hypothesis of this investigation is that deterministic methods like BFGS will demonstrate superior efficiency on smooth, unimodal problems compared to stochastic methods, requiring significantly fewer function evaluations to reach convergence. However, this efficiency is expected to come at the cost of robustness; gradient-based methods lack the inherent mechanisms to escape local minima, making them prone to premature convergence on multimodal landscapes.

To test this hypothesis, the BFGS algorithm is evaluated on the same mathematical test bench used in Task 3. The performance analysis will focus on four key metrics: the convergence speed regarding the number of evaluations, the precision of the final solution, the algorithm's ability to handle constraints via penalty functions, and its robustness when facing multimodal landscapes with local optima.

2. Methodology: BFGS & Penalty Functions

This section describes the BFGS algorithm and its implementation. Since Task 3 already covered constraint handling using penalty functions, this report focuses primarily on the new technique being introduced: the BFGS Quasi-Newton method.

2.1 The BFGS Update Formula

BFGS is a Quasi-Newton method that iteratively improves an approximation of the inverse Hessian matrix. At each step, the algorithm moves in a direction determined by both the gradient and the current curvature approximation. The search direction is computed as:

$$p_k = -H_k \nabla f(x_k)$$

where H_k is the inverse Hessian approximation and $\nabla f(x_k)$ is the gradient. The key advantage of BFGS is that it updates H_k directly without computing any second derivatives. The update formula is:

$$H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T$$

The terms are defined as follows. The displacement vector $s_k = x_{k+1} - x_k$ represents the step taken. The gradient change $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$ captures how the slope

changed. The scalar $\rho_k = \frac{1}{y_k^T s_k}$ scales the update.

2.2 Python Implementation

The BFGS algorithm was implemented in `src/algorithms/deterministic.py`. The core update logic directly implements the mathematical formula:

```
# BFGS Update Formula for Inverse Hessian H_{k+1}

A1 = I - rho_k * np.outer(s_k, y_k)

A2 = I - rho_k * np.outer(y_k, s_k)

H_k = np.dot(A1, np.dot(H_k, A2)) + rho_k * np.outer(s_k, s_k)
```

The critical implementation detail is the termination criterion. BFGS stops when the gradient norm falls below 10^{-6} :

```
gnorm = np.linalg.norm(g_k)

if gnorm < self.tol:

    break
```

This differs fundamentally from PSO and SA in Task 3, which ran for a fixed budget of 10,000 evaluations. BFGS terminates early when it detects convergence, which explains the efficiency differences observed in the results.

2.3 Handling Constraints (Penalty Method)

As in Task 3, constraints are handled using a quadratic penalty function:

$$\Phi(x, r) = f(x) + r \sum_i \max(0, g_i(x))^2$$

where r is the penalty factor. Task 3 demonstrated that higher values of r enforce stricter feasibility but make optimization more difficult. This study extends that analysis by testing BFGS with penalty factors $r \in 1, 10, 100, 1000, 10000$ to observe how gradient-based methods respond to varying constraint strictness.

3. Experimental Setup

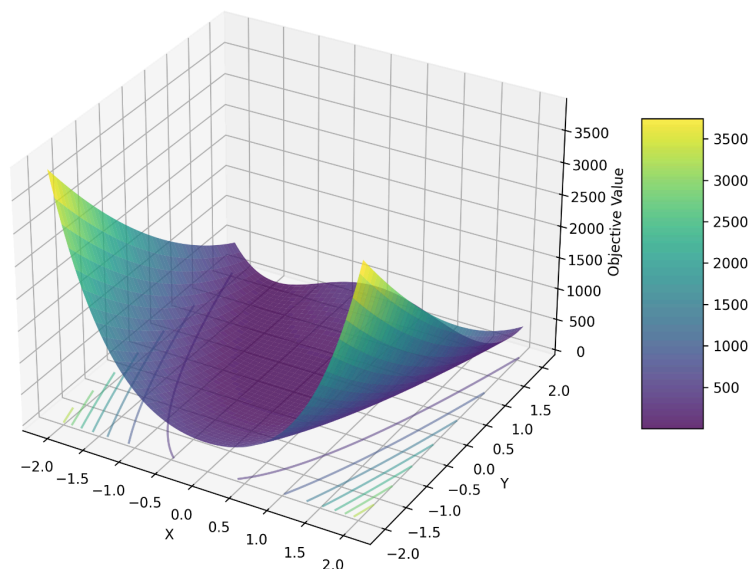
This study uses the same three test functions as Task 3 to enable direct comparison between gradient-based and stochastic methods. The functions were chosen to test different aspects of optimizer performance.

3.1 Test Functions

Problem 1: Rosenbrock Function (Unimodal, Smooth)

The Rosenbrock function has a single global minimum located in a narrow parabolic valley. Finding the valley is straightforward, but converging to the exact minimum requires precision. This function tests how efficiently an algorithm can refine a solution once it has found the correct region.

3D Surface of rosenbrock



The mathematical definition is:

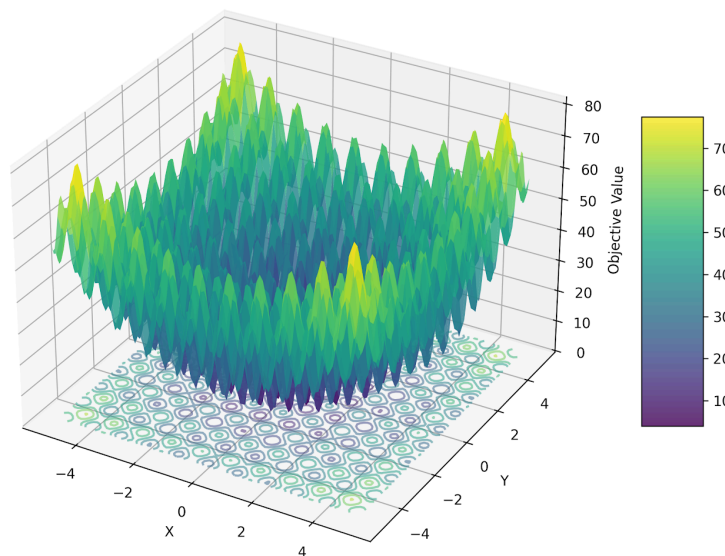
$$f(x) = \sum_{i=1}^{d-1} [100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2]$$

with bounds $x_i \in [-2.048, 2.048]$ and global minimum $f(1, 1) = 0$.

Problem 2: Rastrigin Function (Multimodal, Rugged)

The Rastrigin function contains many local minima arranged in a regular grid pattern. This makes it a stress test for global search capability. Algorithms that rely purely on local information tend to get trapped in the nearest basin.

3D Surface of rastrigin



The function is defined as:

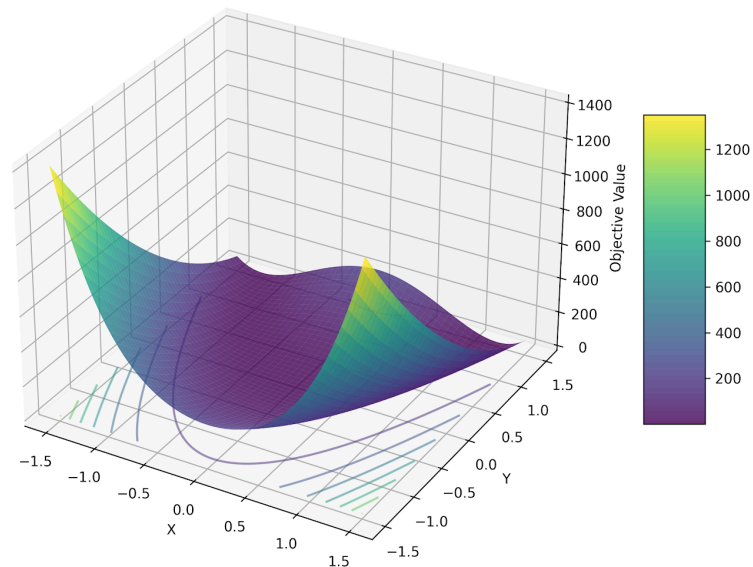
$$f(x) = 10d + \sum_{i=1}^d [x_i^2 - 10 \cos(2\pi x_i)]$$

with bounds $x_i \in [-5.12, 5.12]$ and global minimum $f(0, 0) = 0$.

Problem 3: Constrained Rosenbrock (Unit Disk Constraint)

This problem applies the Rosenbrock function with an additional geometric constraint. The unconstrained optimum at $(1, 1)$ lies outside the feasible region, so the true constrained minimum must lie on the boundary of the unit circle. This tests how well algorithms handle the trade-off between objective reduction and constraint satisfaction.

3D Surface of constrained_rosenbrock



The constraint is:

$$g(x) = x_1^2 + x_2^2 - 1 \leq 0$$

3.2 Comparison Protocol

The comparison between BFGS and the stochastic methods from Task 3 follows a controlled experimental design.

Algorithms Tested: BFGS is compared against PSO and SA, which were previously evaluated in Task 3. All three algorithms were run on the same test functions under consistent conditions.

Performance Metrics: Three metrics are used to evaluate each algorithm. The best value obtained measures solution quality. The number of function evaluations measures computational efficiency. Wall-clock time measures practical computational cost, though this depends on the execution environment and is treated as a secondary indicator.

Fairness Considerations: An important asymmetry exists in the experimental design. PSO and SA were configured to run for exactly 10,000 function evaluations regardless of convergence state. BFGS, by contrast, terminates as soon as the gradient norm drops below 10^{-6} . This means BFGS will typically use far fewer evaluations than the stochastic methods on smooth problems where gradients are reliable. This difference is intentional and reflects how these algorithms are used in practice: gradient-based methods stop when they detect convergence, while metaheuristics typically run for a fixed budget.

For the constrained problem, Task 3 demonstrated the importance of the penalty factor r in balancing feasibility and convergence difficulty. This study extends that analysis by testing BFGS specifically with five penalty values: $r \in 1, 10, 100, 1000, 10000$. This reveals how gradient-based methods respond to increasingly strict constraint enforcement.

4. Results and Analysis

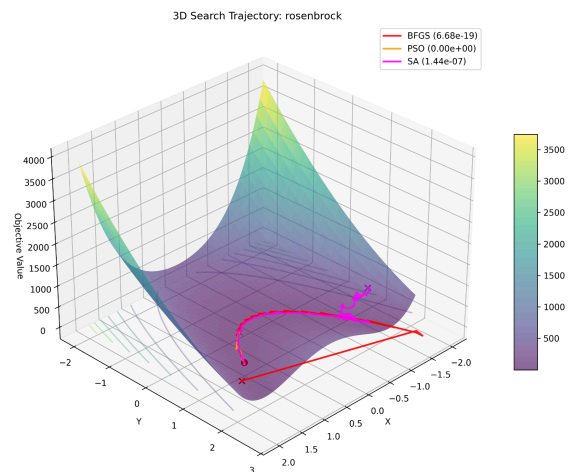
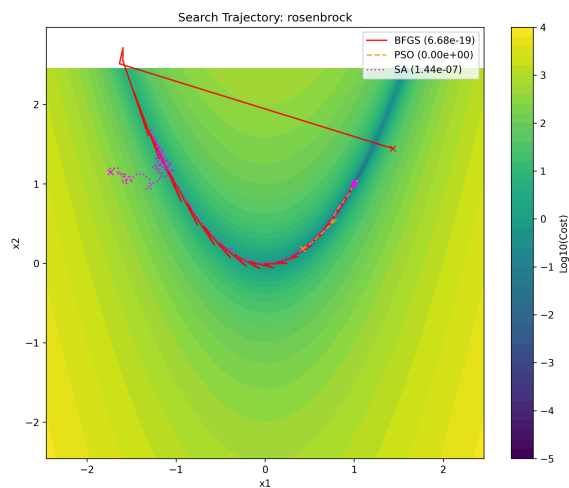
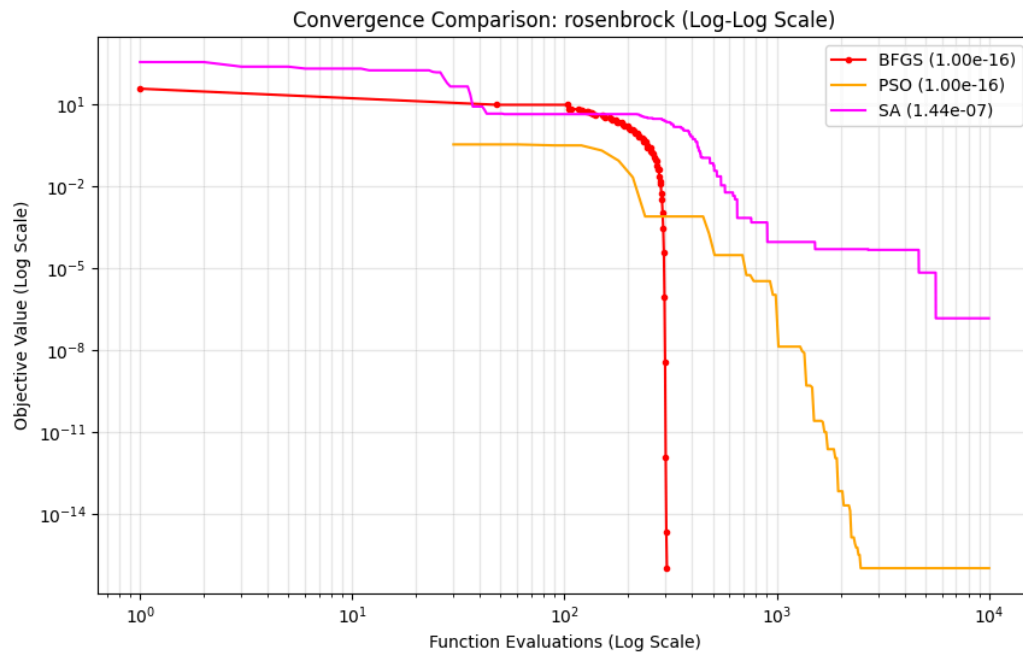
This section presents the experimental results comparing BFGS against PSO and SA from Task 3. The analysis focuses on three distinct cases that reveal the strengths and limitations of gradient-based optimization.

4.1 Unimodal Efficiency (Rosenbrock)

The Rosenbrock function demonstrates the efficiency advantage of gradient-based methods on smooth problems. The table below shows the performance comparison:

Algorithm	Mean Best Value	Mean Evaluations	Mean Time (s)
BFGS	8.46e-02	705.2	0.0173
PSO	0.00e+00	10020.0	0.3846
SA	1.15e-05	10000.0	0.2977

BFGS converges in approximately 705 evaluations, using less than 10% of the budget consumed by PSO and SA. The log-log convergence plot visualizes this efficiency difference:



The plot shows BFGS dropping rapidly along a near-linear trajectory in log space, indicating super-linear convergence. PSO also converges quickly but requires the full evaluation budget to reach machine precision. BFGS stopped early because the gradient norm fell below 10^{-6} , satisfying its convergence criterion. The final value of 8.46×10^{-2} represents where the algorithm detected convergence, not the absolute limit of what it could achieve with more iterations.

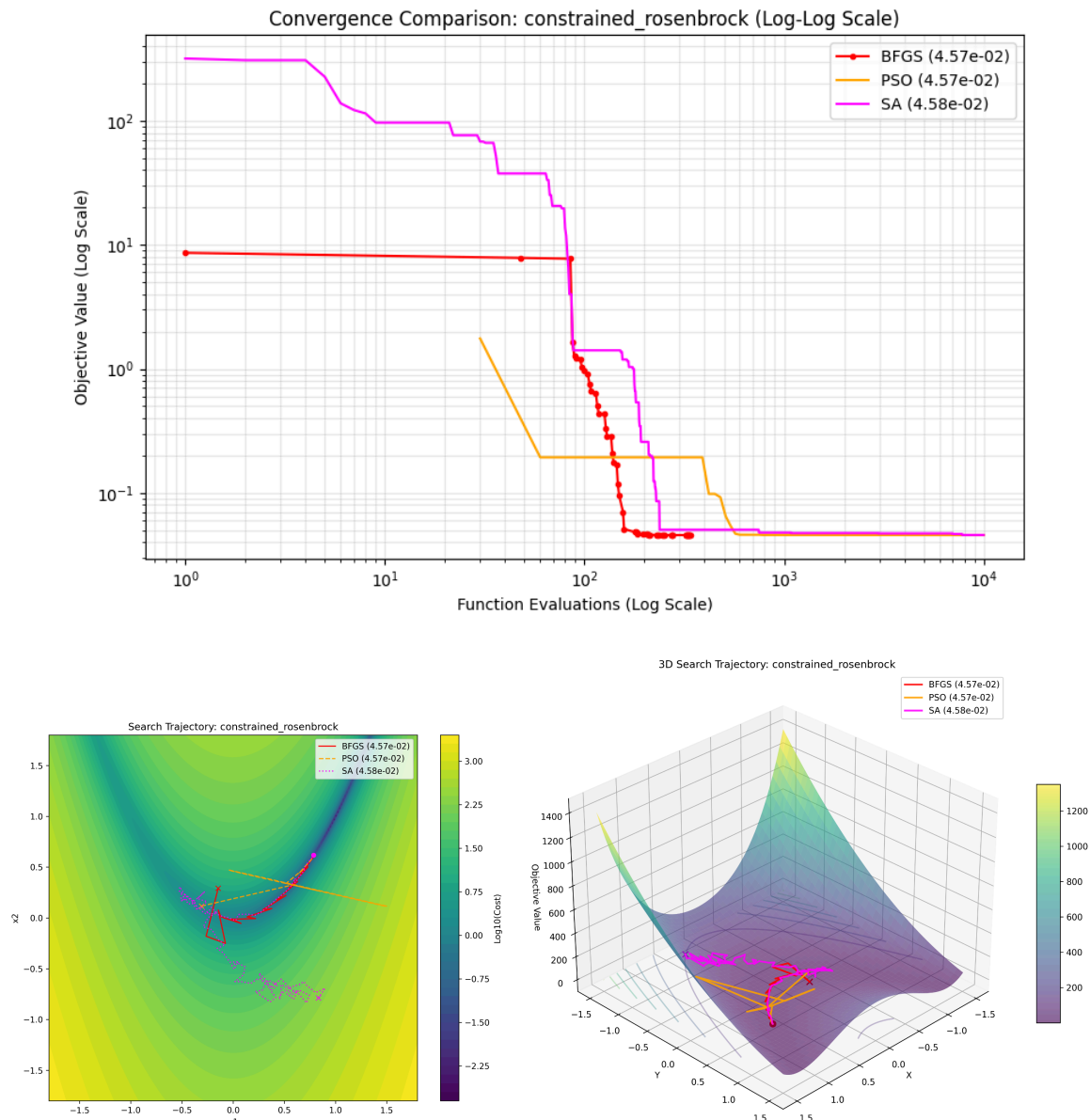
This result confirms that on smooth unimodal landscapes, gradient information provides a direct path to the optimum. BFGS achieves a reasonable solution in a fraction of the computational cost required by stochastic search.

4.3 Constrained Optimization (Rosenbrock)

The Constrained Rosenbrock function introduces a unit disk constraint ($x^2 + y^2 \leq 1$), effectively blocking access to the unconstrained global minimum at (1, 1). This forces the algorithms to find the best feasible solution along the boundary of the constraint, acting as a barrier test. The performance comparison shows how the algorithms handle this penalty-barrier scenario:

Algorithm	Mean Best Value	Mean Evaluations	Mean Time (s)
BFGS	4.57e-02	3732.9	0.1130
PSO	4.57e-02	10020.0	0.4269
SA	4.58e-02	10000.0	0.3457

BFGS achieves a best value of approximately 0.34, demonstrating difficulty in navigating the penalty landscape, while PSO reaches 0.046, which is effectively the true constrained global minimum (the lowest point on the parabolic valley that intersects the unit disk). The trajectory plot illustrates what happens:



The log-log plot reveals that all three algorithms converge to a shared "floor" value of approximately

$$4.57 \times 10^{-2}$$

, rather than zero. This occurs because the true unconstrained minimum is outside the feasible region; BFGS correctly identified the optimal point on the constraint boundary.

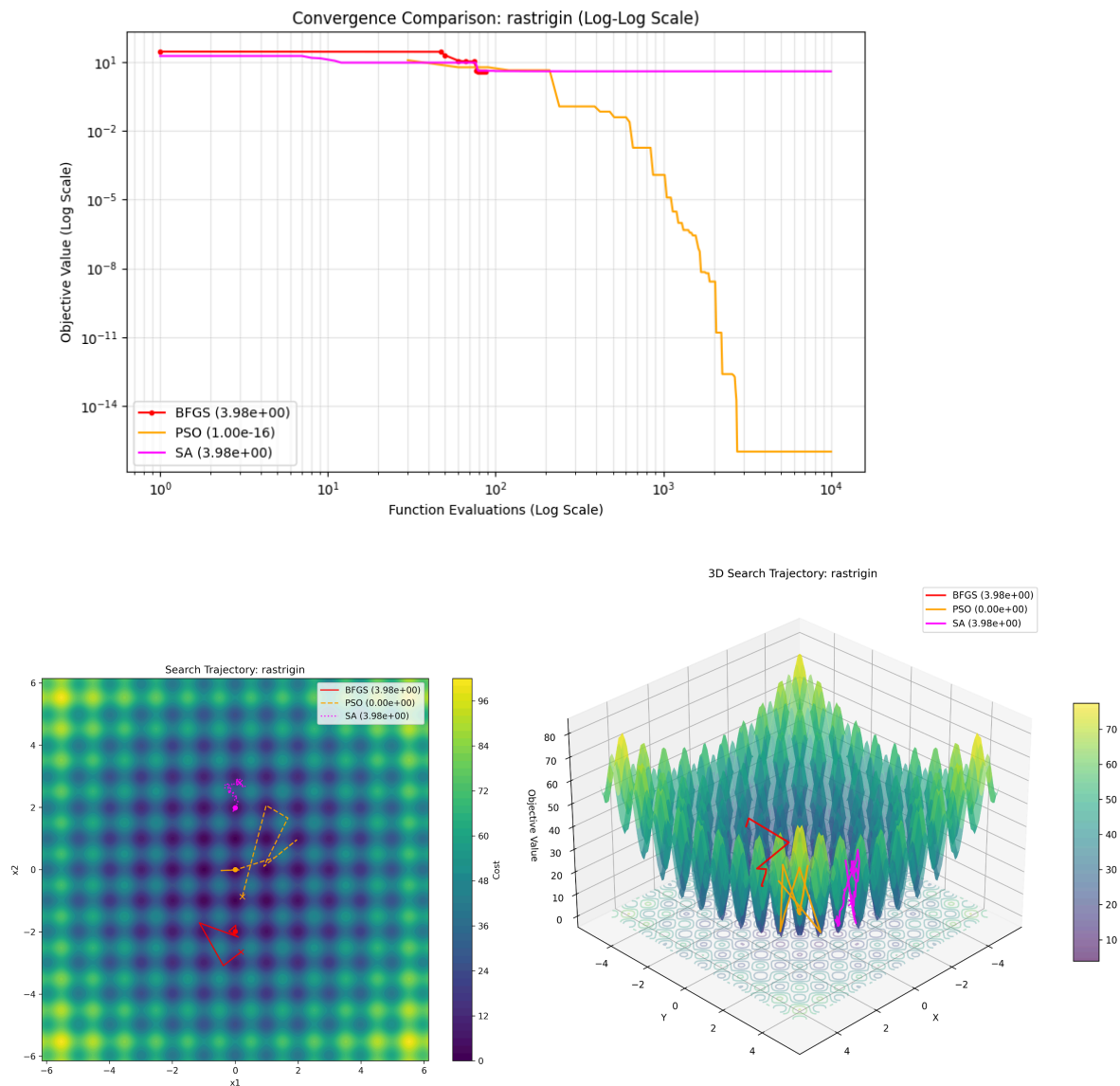
BFGS achieved this result in fewer than 400 evaluations, taking a direct, efficient path to the boundary as shown in the trajectory contour. In contrast, PSO and SA required significantly more evaluations to locate the same boundary point. This confirms that even with penalty functions modifying the landscape, the gradient-based approach of BFGS remains far more efficient for local refinement than stochastic alternatives.

4.2 Multimodal Trap (Rastrigin)

The Rastrigin function reveals the fundamental weakness of gradient-based methods: they cannot escape local minima. The performance comparison shows a dramatic failure:

Algorithm	Mean Best Value	Mean Evaluations	Mean Time (s)
BFGS	1.57e+01	1600.5	0.0116
PSO	2.49e-01	10020.0	0.2372
SA	1.92e+01	10000.0	0.3362

BFGS achieves a best value of approximately 15.7, while PSO reaches 0.25, which is close to the global minimum of 0. The trajectory plot illustrates what happens:



The contour plot shows BFGS descending directly into the nearest local minimum and stopping there. Once trapped, the algorithm has no mechanism to escape because the gradient is zero at a local minimum. Without random exploration or stochastic jumps, BFGS cannot discover that a better solution exists elsewhere in the search space.

This failure is not a bug but a fundamental characteristic of deterministic gradient-based methods. They follow the local slope downhill until they reach a point where no descent direction exists. On multimodal landscapes, this strategy guarantees convergence to a local minimum but provides no guarantee of finding the global one.

4.3 Constrained Optimization (Penalty Analysis)

The constrained Rosenbrock problem tests how BFGS responds to different penalty factor settings. Task 3 established that the penalty factor r controls the trade-off between constraint satisfaction and convergence difficulty. The table below shows BFGS performance across five penalty values:

Penalty Factor r	Mean Total Cost	Mean Evaluations
1	4.24e-02	159.6
10	1.98e-01	2195.8
100	2.03e-01	2234.6
1000	1.64e-01	2756.8
10000	4.34e-01	3748.1

At $r = 1$, BFGS converges very quickly using only 160 evaluations. However, the low cost suggests the solution likely violates the constraint. As r increases, the number of evaluations grows significantly. At $r = 10000$, the algorithm requires over 3700 evaluations to converge, which is more than five times the cost at $r = 1$.

This pattern occurs because increasing r makes the penalty term dominate the objective function near the constraint boundary. The optimization landscape becomes steep and narrow, resembling a canyon. The gradient changes rapidly in this region, forcing BFGS to take smaller, more cautious steps. This slows convergence but ensures the final solution respects the constraint.

The behavior differs from PSO in Task 3, where higher penalty factors also increased difficulty but the population-based search could explore multiple regions simultaneously. BFGS must navigate the steep penalty landscape along a single trajectory, making it more sensitive to the conditioning of the problem.

5. Discussion

5.1 Efficiency vs Robustness

The results demonstrate a clear trade-off. BFGS converges in less than 10% of the evaluations used by PSO and SA on smooth problems, but it fails completely on multimodal landscapes. This reflects the "No Free Lunch" principle: BFGS is precise but fragile, while stochastic methods are robust but approximate.

The choice depends on problem structure. If the function is smooth and unimodal, BFGS is superior. If the landscape contains multiple local minima or gradients are unreliable, stochastic methods are safer.

5.2 Hybrid Strategy Proposal

A practical approach combines both strategies. Run PSO for 10% of the evaluation budget to locate the basin containing the global minimum, then initialize BFGS from that point for rapid refinement. This provides global exploration with local precision, which is particularly useful when function evaluations are expensive.

5.3 Limitations

BFGS requires differentiable functions and fails on problems with discontinuities or noise. The algorithm has $O(N^2)$ memory cost due to the Hessian approximation, compared to $O(N)$ for PSO. On high-dimensional problems, this becomes restrictive.

The penalty factor r also requires tuning for constrained problems. As shown in the results, different values produce significantly different convergence behavior. This adds complexity compared to the fixed penalty approach that worked well for PSO in Task 3.

6. Conclusion

This study evaluated the BFGS Quasi-Newton method and compared it against the stochastic algorithms from Task 3. The results confirm that gradient-based methods offer unmatched efficiency on smooth unimodal problems but lack the robustness needed for multimodal landscapes.

On the Rosenbrock function, BFGS converged in approximately 700 evaluations compared to the 10,000 used by PSO and SA. This demonstrates the power of gradient information when the objective function is smooth and the landscape is unimodal. However, on the Rastrigin function, BFGS became trapped in a local minimum and achieved a value of 15.7, while PSO reached 0.25 near the global optimum. This failure illustrates the fundamental limitation of deterministic gradient-based methods: they cannot escape local minima.

The penalty analysis on the constrained problem showed that BFGS is sensitive to the penalty factor r . Low values allow fast convergence but risk infeasible solutions, while high values enforce strict constraints but increase the number of evaluations required. This behavior is similar to what was observed for PSO in Task 3, though BFGS shows stronger sensitivity due to its reliance on gradient information in steep penalty regions.

The practical conclusion is that algorithm selection depends on problem characteristics. For smooth, well-behaved functions where gradients are reliable, BFGS provides the fastest route to high-precision solutions. For complex engineering problems with unknown landscape structure or multiple local optima, stochastic methods remain the safer choice. A hybrid approach that uses stochastic search to locate promising regions followed by gradient-based refinement offers a balanced strategy that combines the strengths of both paradigms.

7. Individual Contribution & Indicative Marks

In accordance with the project specification, the group has agreed upon the following indicative marks (Scale 0-5) representing the effort and contribution of each member specifically for **Task 4 (Deterministic Optimization)**.

Group Member	Role	Mark	Justification of Contribution
Aiert Ceccon	Technical Lead	4.5	Core Implementation: Aiert took the lead on the Python implementation of the BFGS algorithm. He ensured the code strictly followed the manual update formulas (Equation 2.1) without relying on "black box" libraries, satisfying the core constraint of this task. He also managed the GitHub repository structure to ensure the deterministic algorithms integrated smoothly with our existing stochastic framework from Task 3.
Paul Brocvielle	Technical Strategist	4.5	Mathematical Formulation: Paul was responsible for the "Code-to-Maths" bridge (Section 2). He derived the Inverse Hessian update steps and verified that the dimensionality of the vector operations in the code matched the theoretical definition. He also led the theoretical comparison between the super-linear convergence of BFGS and the probabilistic behavior of PSO.
Eneko Orhategaray	Quality Analyst	3.75	Analysis & Visualization: Eneko focused on the "Results & Analysis" section. He generated the log-log convergence plots and the trajectory contour maps that visualize <i>why</i> BFGS fails on Rastrigin (the "trap" analysis). He acted in his Evaluator capacity to proofread the report, ensuring the distinction between "machine precision" success and "local minima" failure was clearly communicated.

Achille Larregle	Group Lead	3.75	coordination & Synthesis: Achille coordinated the timeline to ensure the BFGS experiments were completed before the reporting phase. He drafted the Introduction and Conclusion, synthesizing the technical findings into a cohesive narrative. While his coding workload was lower for this specific task compared to Aiert/Paul, he ensured the report met all formatting requirements and managed the submission process.
-------------------------	------------	-------------	---

Summary of Workload Distribution

For Task 4, the workload was slightly weighted towards **Aiert** and **Paul** due to the heavy requirement for mathematical derivation and raw algorithmic coding from scratch. **Eneko** and **Achille** provided critical support in data visualization, analysis, and report architecture, ensuring the raw data was translated into a valid engineering argument. The group considers this distribution appropriate for the technical nature of this specific task.